GSoC 2026 Student Application & Proposal

Reviving Mesa-Examples: Bringing Agent-Based Model Examples to Mesa 4.0

Python Software Foundation · Mesa Project · GSoC 2026

Field	Detail
Student Name	Vanya Kapoor
GitHub	https://github.com/Vanya-kapoor
Email	vanyakapoor1010@gmail.com
Phone	+91 9999599647
Country	India
University	Netaji Subhash University of Technology (NSUT), New Delhi
Program	B.Tech Computer Science & Artificial Intelligence, Year 1 (Semester 2)
Preferred Contact	vanyakapoor1010@gmail.com
Mentors	EwoutH (Ewout ter Hoeven), jackiekazil (Jackie Kazil)
Project	Reviving Mesa-Examples
Organization	Python Software Foundation (Mesa)

Table of Contents

1. About Me

1.1 Why I Am the Right Person for This Project

1.2 Software Development Experience

1.3 Previous Open-Source Experience

1.4 Project Links

2. GSoC Background

3. Mesa Community Experience

3.1 Community Interactions

3.2 Code Contributions

3.3 PR Reviews

4. Project Proposal

4.1 Background & Problem Statement

5. Technical Approach & Architecture

5.1 Pillar 1 — Single-File Frontmatter as the Metadata Standard

5.2 Pillar 2 — Example Lifecycle System

5.3 Pillar 3 — Per-Example Isolated CI (Matrix Jobs)

5.4 Pillar 4 — Ownership & Graceful Degradation

5.5 Pillar 5 — AI Discoverability via llms.txt

5.6 Structured 3-Stage Contribution Review Process

5.7 How the Five Pillars Fit Together

6. Primary Goals & Deliverables

Goal 1 — Fix All Broken Examples (Mesa 4.0 Compatibility)

Goal 2 — Metadata, CI & Discoverability Infrastructure

Goal 3 — New Flagship Example: Job Market (Actions API)

Goal 4 — Documentation & Testing

7. Project Timeline

8. Demo — Job Market Model (Mesa 4.0 Actions API)

8.1 Scenario

8.2 Actions API Features Demonstrated

9. Stretch Goals & Further Enhancements

1. About Me

1.1 Why I Am the Right Person for This Project

I am a first-year B.Tech CSAI student at NSUT Delhi with a genuine passion for agent-based modelling, open-source software, and Python. I have been actively participating in coding competitions and building a strong problem-solving foundation. I regularly compete on platforms like Codeforces, CodeChef, and LeetCode, gaining exposure to algorithms and data structures early on. Alongside competitive programming, I have contributed to open source through Mesa, submitting meaningful PRs and working on agent-based models. I continuously explore new technologies, build projects, and learn quickly, show strong initiative, consistency, and the ability to grow rapidly in software development and problem solving.

Over the past several weeks I have immersed myself in the Mesa codebase and its examples repository, performing a systematic audit of all mesa-examples against Mesa 3.x and the Mesa 4.0

pre-release. That audit uncovered six categories of failures — missing dependencies, missing app.py entrypoints, relative import errors, removed APIs, visualization crashes, and DataFrame mismatches — and I have already opened six pull requests fixing the highest-impact issues.

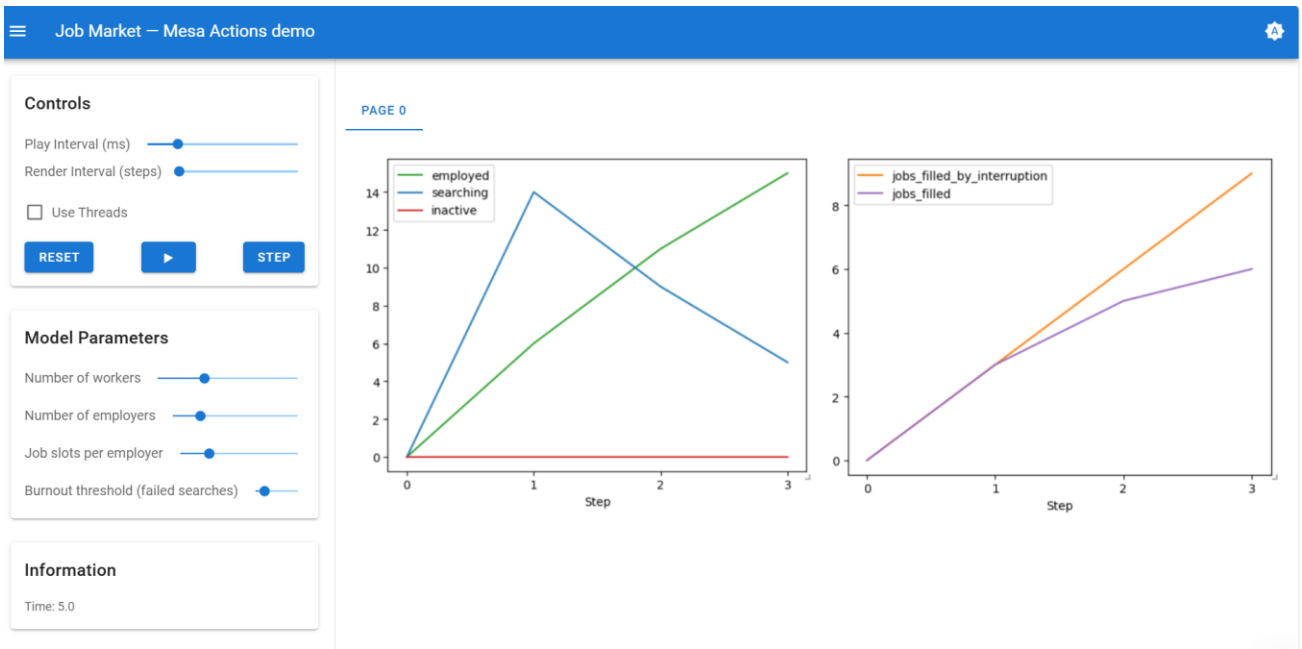
More importantly, I understand the project not just as a maintenance task but as a quality, discoverability, and community-sustainability problem. Broken examples are the first thing a new Mesa user sees. My goal is to make every example run, look good, teach something meaningful, and stay healthy as Mesa evolves.

As a demonstration of deep engagement with the Mesa 4.0 Actions API, I built a complete Job Market discrete-event simulation that exercises Action subclassing, variable-duration callables, priority-based interruption, progress tracking, and the Resource primitive — features that do not yet appear in any mesa-examples model. I have also actively contributed to the architecture discussions in GitHub Discussions #417 and #3272, proposing the llms.txt AI-discoverability layer as a unique contribution.

Image of my audit-

Example	Status	Issue / Notes	Bug Category	Mesa Version	Python Version
aco_tsp	✓ Works	—	—	3.3.1	3.11
bank_reserves	✓ Works	—	—	3.3.1	3.11
boltzmann_wealth_model_network	✓ Works	—	—	3.3.1	3.11
caching_and_replay	✗ Broken	No app.py entrypoint	Missing entrypoint	3.3.1	3.11
charts	✓ Works	—	—	3.3.1	3.11
color_patches	✓ Works	—	—	3.3.1	3.11
conways_game_of_life_fast	✗ Broken	ValueError: Unknown space type: NoneType	Runtime viz crash	3.3.1	3.11
deffuant_weisbuch	✗ Broken	AgentSet.to_list removed	API breaking change	3.3.1	3.11
dining_philosophers	✗ Broken	model.time attribute removed	API breaking change	3.3.1	3.11
el_farol	✗ Broken	No app.py entrypoint	Missing entrypoint	3.3.1	3.11
emperor_dilemma	✗ Broken	ImportError: relative import with no known parent package	Relative import error	3.3.1	3.11
forest_fire	⚠ Warning	3 missing deps: networkx, matplotlib, altair not in requirements.txt	Missing dependencies	3.3.1	3.11
hex_ant	✗ Broken	ImportError: relative import with no known parent package	Relative import error	3.3.1	3.11
hex_snowflake	✗ Broken	ImportError: relative import with no known parent package	Relative import error	3.3.1	3.11
hotelling_law	✓ Works	—	—	3.3.1	3.11
humanitarian_aid_distribution	✗ Broken	SpaceRenderer.setup_agents() removed	API breaking change	3.3.1	3.11
mmc_queue	✗ Broken	No app.py entrypoint	Missing entrypoint	3.3.1	3.11
rumor_mill	✗ Broken	ValueError: pandas DataFrame length mismatch (1024 vs 993)	Data collector bug	3.3.1	3.11
shape_example	✗ Broken	No app.py entrypoint	Missing entrypoint	3.3.1	3.11
termites	✗ Broken	TypeError: add_property_layer() takes 2 args but 3 given	API breaking change	3.3.1	3.11
virus_antibody	✗ Broken	AttributeError: NoneType has no attribute agent_positions	Runtime viz crash	3.3.1	3.11
warehouse	✗ Broken	AttributeError: AgentSet.to_list removed	API breaking change	3.3.1	3.11

Simulation of my job market model



1.2 Software Development Experience

- Python (primary): Mesa ABM, Solara, pandas, NumPy, matplotlib, altair
- C++: competitive programming (Codeforces, CP31 curriculum)
- Web: HTML/CSS/JavaScript
- Tools: Git, GitHub Actions CI, conda, pip, pytest

- Built a complete Mesa 4.0 Actions API demonstration model (Job Market) from scratch
- Performed systematic audit of 20+ mesa-examples against Mesa 3.3.1 and Mesa 4.0 pre-release
- Wrote CI validation pipeline (GitHub Actions) for metadata.toml consistency across all examples
- Implemented metadata.toml reference standard for mesa-examples discoverability
- Authored llms.txt for AI/LLM discoverability of mesa-examples — proposed and discussed in GitHub Discussions #417

1.3 Previous Open-Source Experience

Mesa (projectmesa/mesa-examples) is my first open-source contribution. Before GSoC I had zero open-source experience; everything I have learned about contributing — forking, branching, PR etiquette, CI, code review — has come from working on Mesa over the past few months. My understanding of the Mesa codebase is entirely self-built and genuine.

1.4 Project Links

- GitHub profile: https://github.com/Vanya-kapoor

- mesa-examples PRs: https://github.com/mesa/mesa-examples/pulls?q=is%3Apr+author%3AVanya-kapoor

- Job Market model (GSoC demo): https://github.com/Vanya-kapoor/GSoC-learning-space

- GitHub Discussions contribution: https://github.com/mesa/mesa-examples/discussions/417
- https://github.com/mesa/mesa/discussions/3304

2. GSoC Background

- Other commitments: My second-semester university exams fall approximately April end-15 May 2026. I have planned the project timeline to front-load community bonding work and keep exam weeks as buffer/review weeks. No internship, travel, or other major commitments during the GSoC coding period.
- Previous GSoC participation: No. This is my first GSoC application.
- Mentoring another organization: No.
- Applying to multiple organizations: No. Mesa / Python Software Foundation is my sole application.

- Eligibility: Yes. I have reviewed the GSoC 2026 eligibility criteria (section 7.1) and confirm that I meet all requirements.

3. Mesa Community Experience

3.1 Community Interactions

- Matrix chat (mesa-dev channel): regular presence; discussed mesa-examples audit findings, Actions API design questions, and metadata standard with EwoutH and jackiekazil
- Also talked to Aman Bihari,Ved Prakash,Harshini,Darragh Moran
- GitHub Discussions #3272 (Metadata Standard): contributed to the converged metadata schema discussion with jackiekazil and areyoush; proposed the frontmatter-first approach
- GitHub Discussions #417 (Revival Planning): contributed the llms.txt AI-discoverability concept and frontmatter pivot (PR #425); greenlit by EwoutH
- GitHub Discussions #3272: EwoutH explicitly greenlit my Job Market model idea as a strong Actions API demonstration
- Reviewed peer contributions including continuous predator-prey (#347) and SIR model (#435), providing structured feedback

Engagement with the Development Activities of Mesa

I have been participating in architectural discussions happening with Mesa developing on the current Actions API and have acted as a resource within these discussions.

Discussion #3591, led by Ewout H, introduced lifecycle hooks (on_action_complete, on_action_interrupt) to facilitate identifying the completion of actions at the agent level in Mesa. I made suggestions for making the API clearer and for creating a more modular architecture by developing a HasActions mixin to isolate agent-level functionality from action-level functionality, while also enabling agents to be extensible without having to modify base Agent functionality.

I also had the opportunity to review the architecture codebreaker32 proposed using a more decoupled architecture, consisting of a TaskManager + signal-based behavioral framework. This architecture separates:

-The actual Effects of Actions being made in an Action, i.e., what happens during action execution (within the Action Class)

-Decision Making, (an Action), performed by a separate System that observes the Signal created by an order of action completion.

I feel that these experiences are providing me with a greater understanding of:

-The Separation of Concerns on agent based systems,

-Two different ways to attach code to an event on a synchronous basis and event driven fashion

-Designing APIs that are extensible/maintainable

I plan to use these lessons to guide the future development of my examples in the mesa-examples repository (i.e., design examples that teach modern mesa constructs, including the Actions API and modular agent behavior).

3.2 Code Contributions

Issue / Context	PR Link	Status
Relative import errors across multiple examples	mesa-examples #427	Open / Pending Review
Missing app.py entrypoints	mesa-examples #428	Open / Pending Review
forest_fire notebook Mesa 4 migration	mesa-examples #422	Open / Pending Review
metadata.toml reference implementation	mesa-examples #424	Open / Pending Review
CI metadata validation (GitHub Actions)	mesa-examples CI PR	Open / Pending Review
llms.txt for AI discoverability	mesa-examples #425	Open / Pending Review
TOTAL	6 PRs	

3.3 PR Reviews

Project	PR Link	Key Feedback
mesa-examples	#347 (continuous predator-prey)	Flagged missing Mesa 4 AgentSet migration; suggested replacing deprecated schedule.agents with agents_by_type; noted missing app.py entrypoint
mesa-examples	#435 (SIR model)	Identified relative import error; recommended metadata.toml addition; suggested DataCollector reporter alignment with Mesa 4 API
mesa-examples	self-initiated audit	Documented 6 failure categories across 20+ examples; created structured bug taxonomy used as PR roadmap
TOTAL	3 reviews	

4. Project Proposal

4.1 Background & Problem Statement

Mesa is the leading Python framework for agent-based modelling (ABM), with thousands of researchers and students relying on mesa-examples as their primary entry point. However, mesa-examples has not kept pace with Mesa's rapid evolution from 2.x through 3.x to the forthcoming 4.0 release. My pre-GSoC audit revealed that the majority of examples fail to run out of the box, falling into six distinct failure categories:

- Missing dependencies (requirements.txt absent or incomplete)
- Missing app.py entrypoints (Solara visualization not launchable)
- Relative import errors (broken since Mesa restructured its package layout)
- Removed APIs (schedule, time_step, deprecated Mesa 2/3 patterns)
- Visualization crashes (SolaraViz API changes, missing portrayal updates)
- DataFrame mismatches (DataCollector reporter signatures broken by Mesa 4 changes)

Beyond technical breakage, mesa-examples suffers from three structural problems surfaced in the maintainer discussions (GitHub Discussions #417 and #3272): silent rot (no CI to detect breakage), poor discoverability (new users cannot filter by domain, complexity, or Mesa feature), and contributor friction (no standards, no review process, quality varies wildly). This proposal addresses all three layers.

5. Technical Approach & Architecture

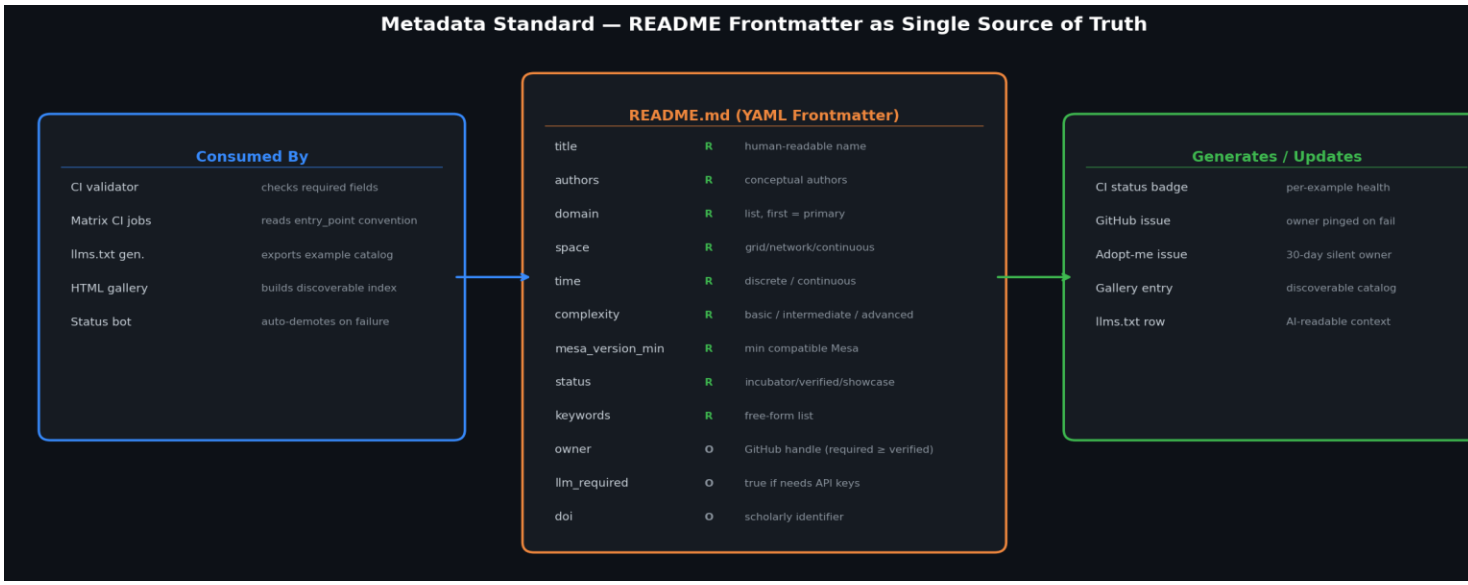
My approach synthesizes the best ideas from the maintainer and community discussions in GitHub Discussions #417 and #3272 into a coherent, layered system. The five pillars below are designed to work together: each addresses a different failure mode, and together they create a self-sustaining ecosystem.

5.1 Pillar 1 — Single-File Frontmatter as the Metadata Standard

Rather than maintaining a separate meta.yaml and a README.md that overlap heavily (as debated in early discussion), I adopt the pattern proposed by @EwoutH: a single README.md with YAML frontmatter. This is a well-established pattern (Hugo, Jekyll, Quarto, Pandoc all use it), so contributors will recognize it immediately and tooling already exists.

```
title: Forest Fire Model
authors: [Per Bak, Kim Christensen]
domain: [ecology, complex-systems]      # first = primary
space: grid                             # grid / network / continuous
time: discrete                          # discrete / continuous
complexity: basic                       # basic / intermediate / advanced
mesa_version_min: "3.0"
status: incubator                       # incubator / verified / showcase / deprecated
keywords: [forest-fire, self-organized-criticality, cellular-automaton]
owner: null                             # required for verified and above
llm_required: false                     # true if example needs API keys
doi: null                               # optional scholarly identifier
```

This draws on the converged metadata standard from GitHub Discussion #3272, where jackiekazil and areyoush aligned on required fields: title, authors, domain, space, time, complexity, mesa_version_min, status, keywords, and doi (optional). The key insight: one file, no drift between metadata and documentation. CI parses the frontmatter block, validates required fields against the declared status level, and treats everything below the second --- as human-readable documentation.



5.2 Pillar 2 — Example Lifecycle System

Every example carries an explicit status that reflects its maturity. Status makes quality visible to users, sets clear expectations for contributors, and gives CI a handle for scaling validation depth:

Status	Meaning	Required Fields	Owner Required	CI Depth
incubator	Works, not yet polished. Open to all contributors.	All frontmatter fields	No	Runs N steps without error
verified	Reviewed, documented, actively maintained.	All fields + owner	Yes	N steps + DataCollector non-empty
showcase	Editorially selected as exemplary.	All fields + owner + maintainer OK	Yes	Full suite + notebook
deprecated	Broken or unmaintained. Visible to users as such.	title + status only	No	None (issue opened)

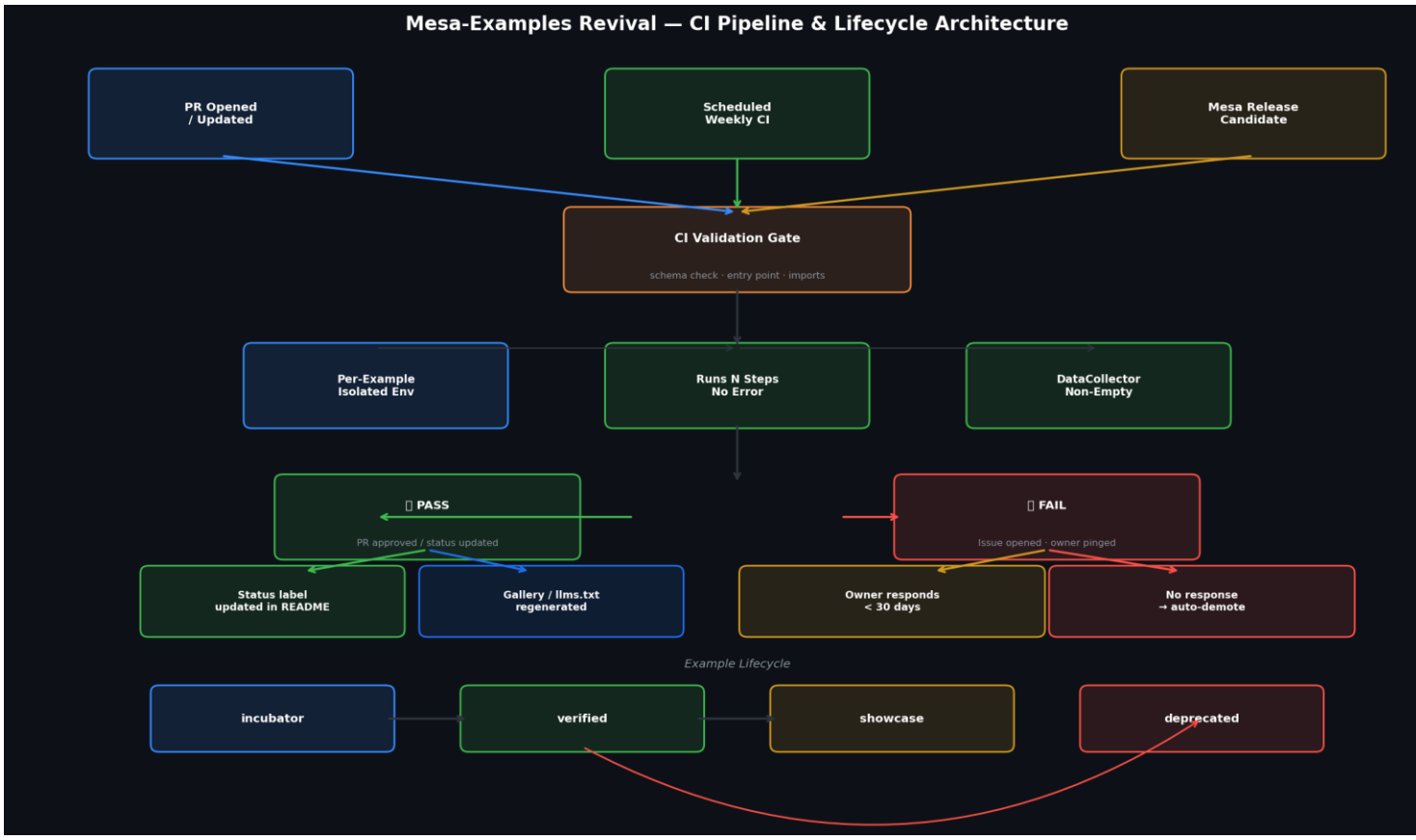
Status lives in the README frontmatter (the single source of truth). GitHub labels mirror the metadata status as a view helpful for filtering PRs, but CI updates the label from the file — never the other way around. This was the key design decision from the Discussion #417 thread.

5.3 Pillar 3 — Per-Example Isolated CI (Matrix Jobs)

A shared requirements.txt across all examples (suggested by some contributors) breaks examples that depend on different versions of the same library. The correct solution — confirmed by consensus in Discussion #417 — is a GitHub Actions matrix job where each example gets its own isolated environment.

- Convention-first entry points: CI looks for app.py, then run.py, then falls back to a declared entry_point field. 90% of contributors never need to think about it.
- Auto-generated pinned requirements.txt from pip freeze scoped to the example's imports on first contribution. CI installs from this lockfile — reproducible, no shared state.

- Per-example validation: (1) does it run N steps without error, (2) is DataCollector model-level output non-empty, (3) is DataCollector agent-level output non-empty. Three checks catch the vast majority of real breakage.
- LLM examples (llm_required: true): CI uses a lightweight mock LLM responder that returns a fixed string, testing that Mesa machinery works even without API keys.
- Cross-check: if requirements.txt pins mesa==3.x but mesa_version_min frontmatter says 4.0, CI flags the inconsistency automatically.



5.4 Pillar 4 — Ownership & Graceful Degradation

Every verified example has an explicit owner: the person responsible for keeping it healthy. Inspired by @Nandha-kumar-S's proposal and the Discussion #417 thread, ownership is tied together with CI and the CODEOWNERS file:

- Ownership commitment: respond within 14 days when a CI-opened issue tags you, and either fix it or explicitly hand off. Bounded and explicit — not a maintenance burden.
- When an example fails scheduled CI, a bot opens a GitHub issue tagged status: broken and pings the CODEOWNERS-declared owner via @ mention.
- If there is no fix or response within 30 days, an automated script changes the README frontmatter from status: verified to status: deprecated and opens a call-for-adoption issue.
- Incubator examples do not require an owner, keeping the barrier to entry low.

This ensures the repository degrades gracefully rather than silently — when people move on, the system makes the state visible and actionable.

5.5 Pillar 5 — AI Discoverability via llms.txt

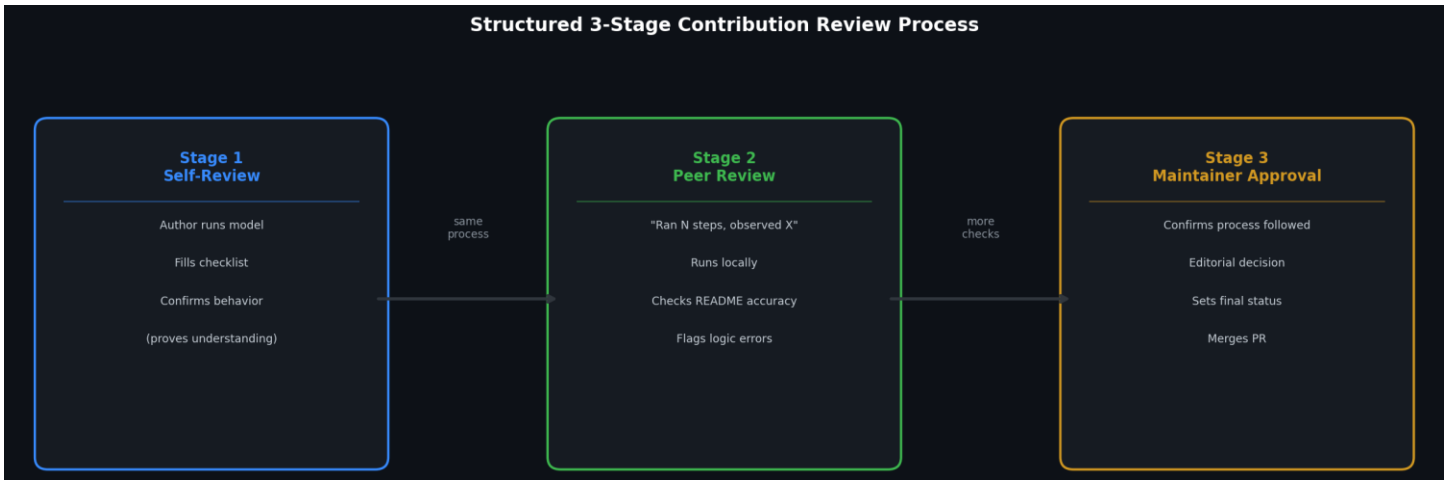
Human discoverability is solved by the gallery/catalog generated from frontmatter metadata. But AI discoverability is a separate and underserved layer. When a user asks Claude or Copilot "how do I build a Mesa model?", the AI either uses stale training data (pre Mesa 3.x) or scrapes HTML and gets noise — recommending broken examples to new users, which is one of the most common friction points for people discovering Mesa.

My unique contribution to this project — proposed in Discussion #417 and implemented in PR #425 — is an llms.txt at the repository root: plain Markdown that gives AI tools accurate Mesa 3.x/4.x context, the full working example list, and a migration error table in one fetch. It is complementary to the frontmatter system:

- Frontmatter feeds the human-readable catalog (HTML gallery, GitHub labels, README index)
- llms.txt feeds AI tools: Claude, Copilot, Cursor, and other AI coding assistants that respect the llms.txt standard
- The broken-example health table in llms.txt means AI tools stop recommending deprecated examples — automatically updated by CI on every scheduled run

5.6 Structured 3-Stage Contribution Review Process

A structured review process distributes maintainer load and builds community quality standards — especially important in the era of AI-generated code, where self-review is how contributors demonstrate they understand what they're submitting. Proposed by @abhinavk0220 in Discussion #417 and refined here:



- Stage 1 — Self-review: author runs the model and fills a checklist. The minimum bar: "I ran the model for N steps and observed [specific emergent behavior]." Cannot be faked without understanding the model.
- Stage 2 — Peer review: one reviewer (incubator) or two (verified) must confirm they ran the model locally and describe observed behavior. The review template requires this sentence — incomplete reviews are flagged automatically by the PR checklist.

- Stage 3 — Maintainer approval: confirms the process was followed and makes the editorial call (promote to verified/showcase, or keep at incubator). Same three-stage flow at every tier — just more checklist boxes as the bar rises.

Review depth scaling: Incubator requires 1 reviewer + basic behavior observation. Verified requires 2 reviewers + current Mesa main compatibility check + README factual accuracy review. Showcase requires maintainer sign-off in addition to the above.

5.7 How the Five Pillars Fit Together

The pillars are not independent features — they form a feedback loop. A new contributor submits a PR: the frontmatter template guides them. CI validates the schema and runs the model. If it passes, the status is set to incubator and the gallery and llms.txt are regenerated. When a maintainer promotes an example to verified, an owner is declared. If that example later breaks under a new Mesa release, the scheduled CI detects it, pings the owner, and — if unresolved — gracefully demotes the example and opens a call-for-adoption. At every step, quality is visible and actionable without requiring constant maintainer attention.

6. Primary Goals & Deliverables

Goal 1 — Fix All Broken Examples (Mesa 4.0 Compatibility)

Deliverable: Every example in mesa-examples runs with pip install mesa (4.0 release) and solara run app.py with zero errors.

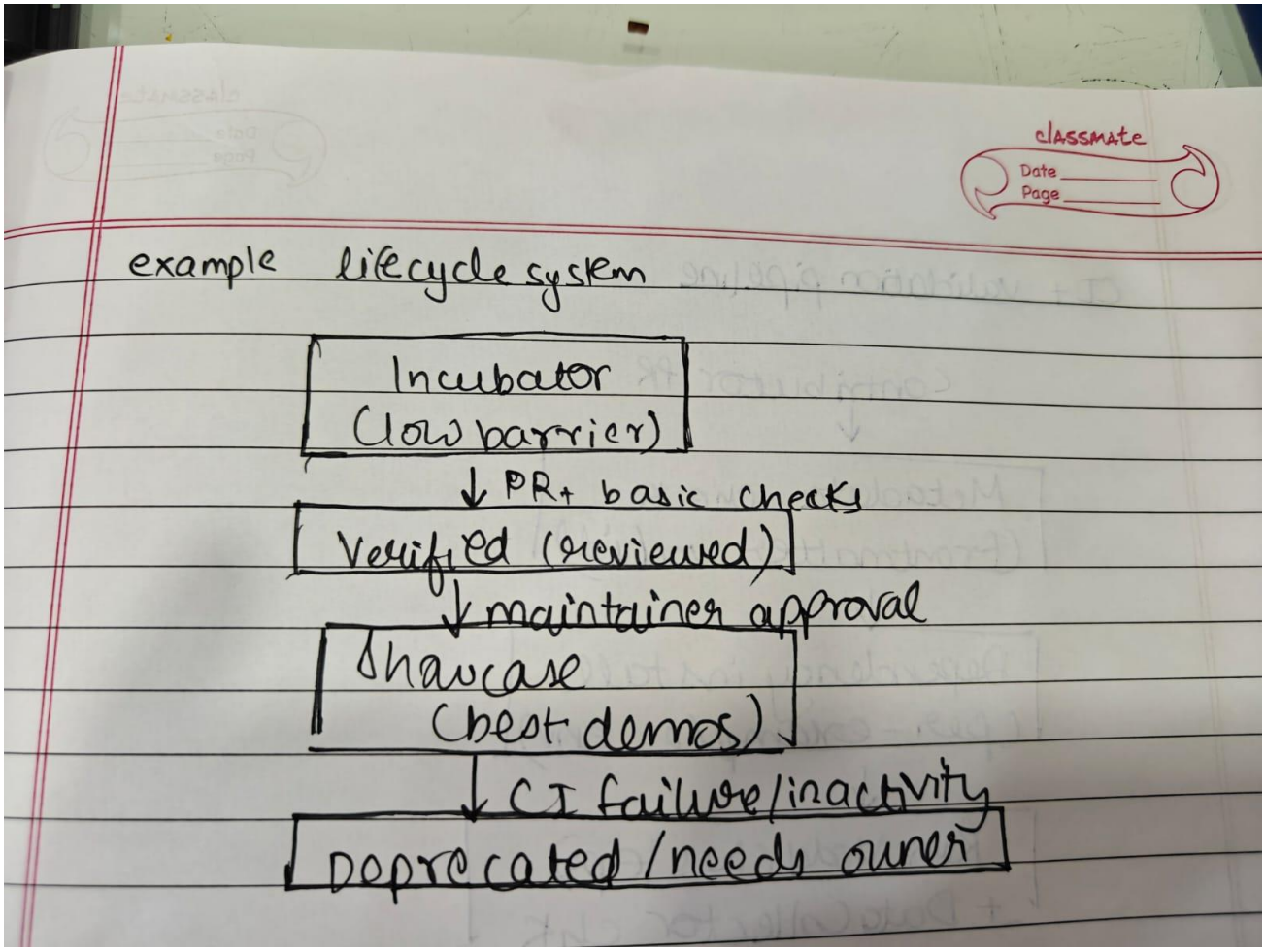
- Migrate all examples from deprecated scheduler API (RandomActivation, SimultaneousActivation) to AgentSet.shuffle_do() / do()
- Fix all relative import errors across the repository
- Add or fix app.py entrypoints for every example with a visualization
- Update DataCollector reporter signatures broken by Mesa 4 changes
- Update SolaraViz usage to current API (model instance, correct param passing)
- Add requirements.txt / pyproject.toml to every example missing dependencies

Goal 2 — Metadata, CI & Discoverability Infrastructure

Deliverable: Every example ships a README with YAML frontmatter following the converged standard from Discussions #417 and #3272, plus CI validation that fails on missing or malformed metadata.

- Finalize README frontmatter schema with all five pillar components in place
- Add frontmatter to all existing examples (migrated from metadata.toml where already present)
- Ship GitHub Actions CI: PR validation (schema check) + scheduled weekly runs (compatibility testing)
- Auto-generate and deploy HTML gallery from frontmatter metadata to GitHub Pages
- Ship llms.txt at repository root for AI-tool discoverability
- Implement ownership + graceful degradation: CODEOWNERS integration + auto-demote bot

1. Example Lifecycle Chart

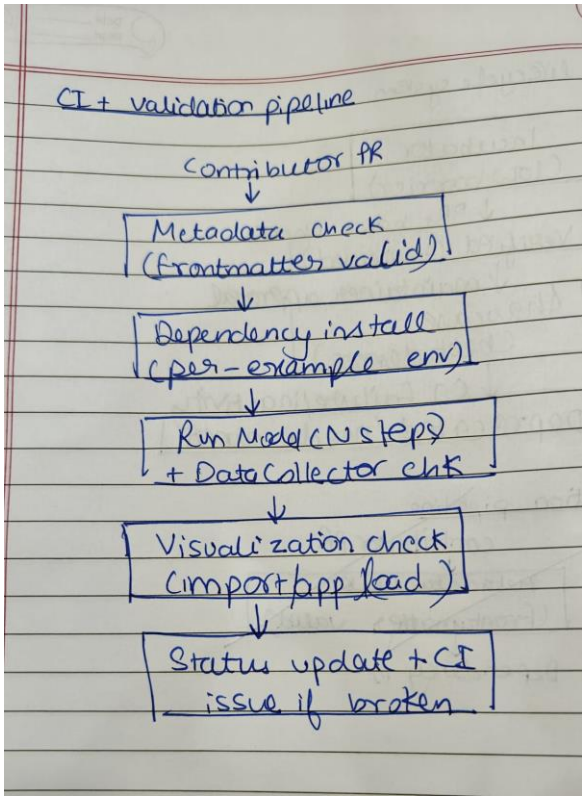


This lifecycle is designed to solve a tension I noticed while working with mesa-examples: making contributions easy often leads to lower quality, while strict standards discourage contributors.

By introducing an *Incubator → Verified → Showcase* progression, we separate contribution from curation. Contributors can submit freely without being blocked by strict requirements, while maintainers retain control over which examples are promoted.

An important addition here is the possibility of *demotion*. Examples that break due to API changes or dependency issues can move back from Verified to Deprecated, ensuring the repository reflects the current state of reliability rather than past quality.

2. CI + Validation Pipeline



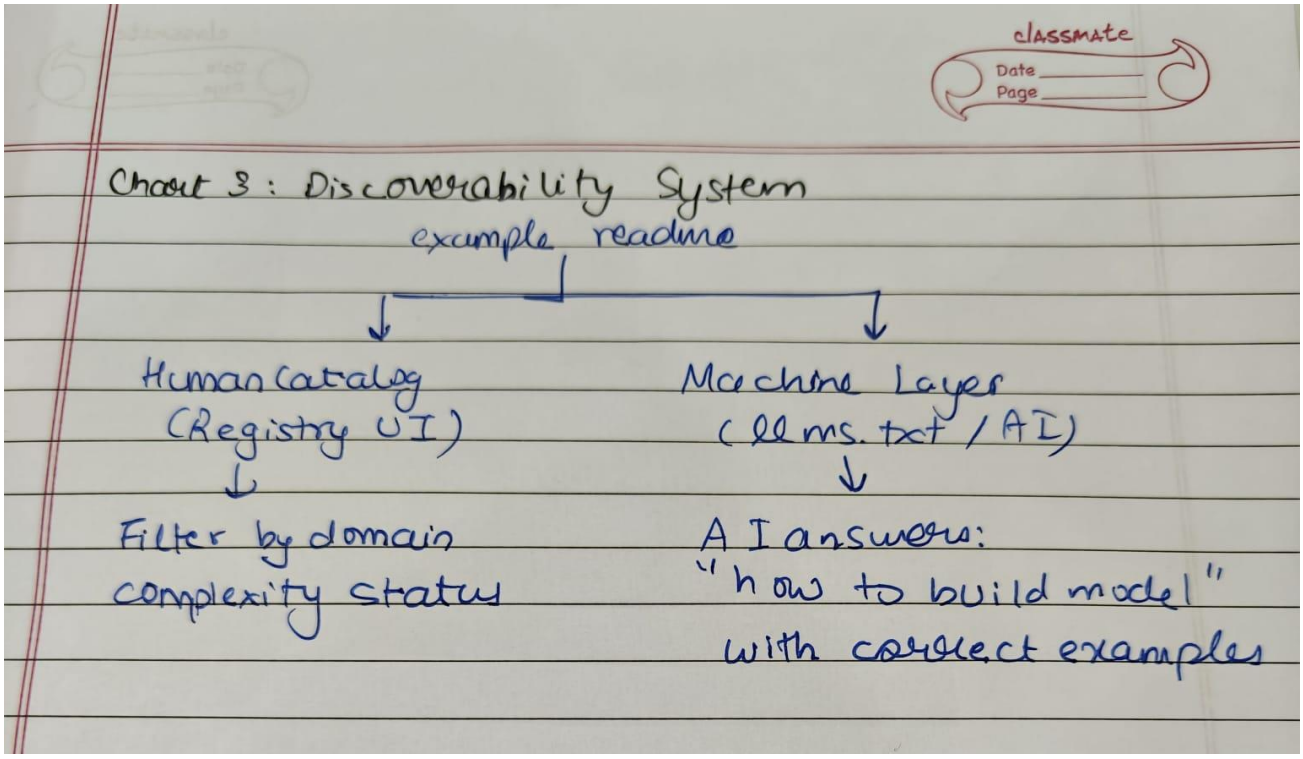
While debugging examples, I found that many failures were not visible unless the model was run manually. Some examples would import correctly but fail after a few steps, while others would silently produce incorrect outputs.

This pipeline is designed to convert those silent failures into visible signals. Instead of relying on manual testing, each example is automatically validated through a sequence of checks — from metadata validation to execution and visualization loading.

The “run for N steps” check is particularly important, as it ensures that the model is not just syntactically correct, but functionally stable.

One challenge here is testing visualization reliably, especially with Solara. My plan is to initially validate import and app initialization, and iterate toward more robust checks based on feasibility and mentor feedback.

3. Discoverability System



While exploring mesa-examples, I noticed that even when examples worked, they were difficult to navigate — especially for beginners trying to find relevant models.

This system introduces structured metadata (via README frontmatter) that enables both human and machine discoverability. Users can filter examples by domain, complexity, and status, while the same structured data can power an AI-friendly layer through llms.txt.

This dual-layer approach ensures that examples are not only easier to browse manually, but also correctly surfaced by AI tools when users ask questions like “how to build a predator-prey model in Mesa.”

I believe this is an important step toward making mesa-examples not just a repository, but a learning interface.

Goal 3 — New Flagship Example: Job Market (Actions API)

Deliverable: A complete, well-documented, fully-tested Job Market agent-based simulation — the canonical demonstration of Mesa 4.0's experimental Actions API.

- Action subclassing: SearchForJob and BurnOut both extend Action
- on_start, on_complete, on_interrupt(progress) hooks all exercised
- Variable-duration actions via callable: duration=lambda a: max(1, 10 - a.skill * 2)
- Priority-based interruption: employers hire highest-skill searching workers first
- Resource primitive: job slots as a shared Resource with capacity
- Full Solara visualization with employment/search/burnout time-series charts

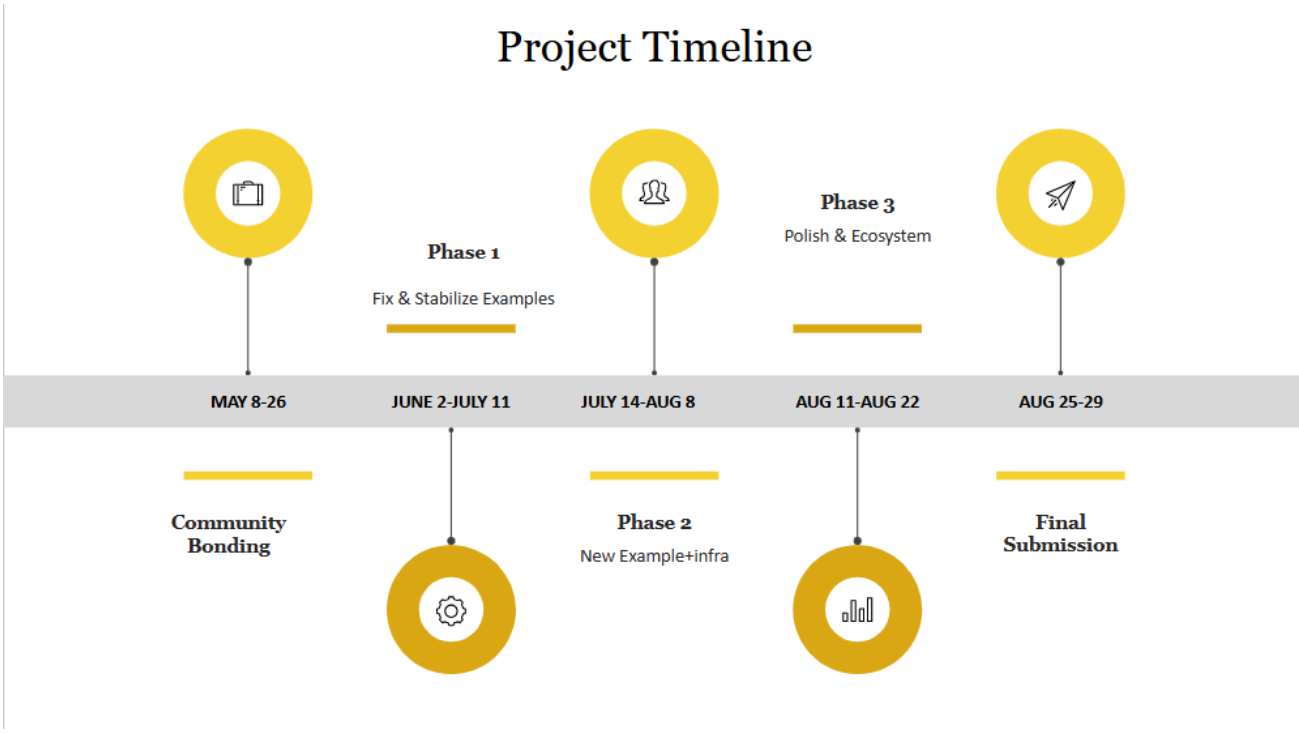
- Comprehensive README frontmatter, pytest smoke tests, and Jupyter notebook walkthrough

Goal 4 — Documentation & Testing

Deliverable: Each migrated or new example ships with an updated README (mini-paper structure), at least one pytest smoke test, and a notebook walkthrough for key examples.

- Smoke tests: model instantiates and runs 10 steps without error; DataCollector non-empty
- README mini-paper structure (from Discussion #417): abstract, background, model description, how to run, results & discussion, references
- Notebook walkthroughs: step-by-step explanation suitable for teaching (3–5 key examples)

7. Project Timeline



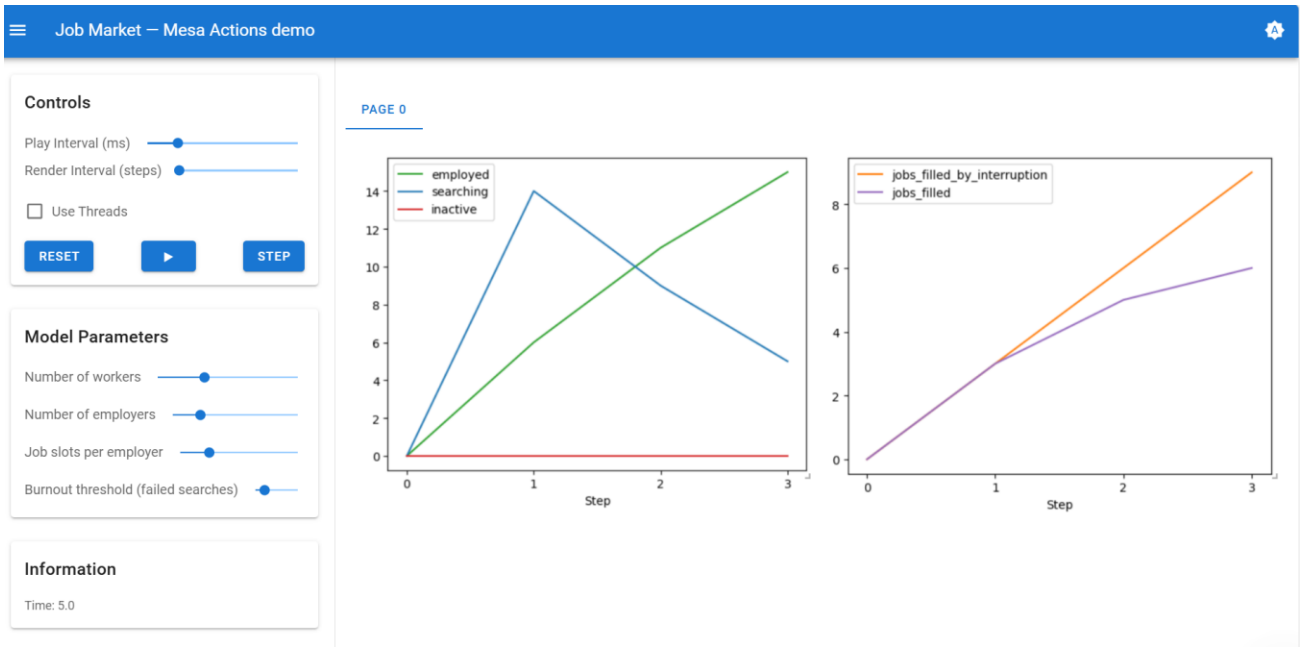
Period	Focus	Detailed Work	Deliverables	Risk Mitigation
Community Bonding (May 8–26)	Planning & Alignment	Review Discussions #417, finalize frontmatter schema, merge PRs, audit all examples	Final schema + prioritized example list + CI skeleton	Buffer for exams
Week 1–2	Batch 1 Fixes	Fix 8 most-used examples (imports, APIs, app.py, DataCollector)	8 working examples + frontmatter + tests	Start with high-impact examples

Period	Focus	Detailed Work	Deliverables	Risk Mitigation
Week 3–4	Batch 2 Fixes	Fix next 8 examples + integrate metadata CI validation	16 total working examples + CI validation live	Parallelize fixes
Week 5–6	Full Stabilization	Fix remaining examples, edge cases, ensure all CI passes	100% examples working on Mesa 4.0	Buffer for exams/review
Week 7–8	Job Market Model (Core)	Implement Actions API features, write pytest suite	Fully functional model.py + tests	Independent module (low dependency risk)
Week 9–10	Visualization + Docs	Solara app, README mini-paper, notebook, llms.txt	Complete flagship example + AI discoverability	Iterate with mentor feedback
Week 11–12	Ecosystem Build	Gallery deployment, ownership system, auto-demotion bot	Sustainable infra + GitHub Pages gallery	Scope stretch if needed
Final Week	Wrap-up	Final fixes, documentation, submission	Final report + merged PRs	Buffer week

Total working hours: ~350 hours across 13 weeks (~27 hrs/week). I am available full-time during the summer.

8. Demo — Job Market Model (Mesa 4.0 Actions API)

I have built a complete Job Market discrete-event simulation available at: https://github.com/Vanya-kapoor/GSoC-learning-space/tree/main/models/job_market



8.1 Scenario

Workers search for jobs. Search duration varies by skill level — skilled workers find jobs faster (duration = max(1, 10 - skill * 2)). Employers hold a fixed number of job slots. When a slot opens, the employer scans all currently-searching workers and interrupts the highest-skill worker, hiring them immediately. Workers who fail too many consecutive searches burn out and become inactive.

8.2 Actions API Features Demonstrated

- Action subclassing: SearchForJob and BurnOut both extend Action
- on_start hook: sets worker status to "searching", increments search counter
- on_complete hook: worker found job independently; increments jobs_filled_independently
- on_interrupt(progress) hook: partial search credit added to experience; employer sets status to "employed"
- duration as callable: lambda a: max(1.0, 10.0 - a.skill * 2.0)
- priority as callable: lambda a: a.skill (higher skill = hired first)
- interruptible=False on BurnOut: once burning out, cannot be hired
- ActionState.ACTIVE check: Employer only interrupts genuinely active searches
- Resource primitive: job slots modelled as a shared Resource with capacity
- model.run_for(1.0): Mesa 4.0 stable time advancement API

8.3 Existing Pre-GSoC PRs (Proof of Engagement)

- #427 — Fix relative imports across multiple examples
- #428 — Add missing app.py entrypoints
- #422 — Migrate forest_fire notebook to Mesa 4
- #424 — metadata.toml reference implementation
- #425 — llms.txt for AI discoverability
- CI PR — GitHub Actions metadata validation

9. Stretch Goals & Further Enhancements

- Second new example: Discrete-event hospital simulation using Resource + Action to model patient admission queues — another strong Actions API showcase
- Auto-generated HTML gallery deployed to GitHub Pages from frontmatter metadata
- Notebook-first examples: convert 3–5 key examples to Jupyter notebooks with step-by-step narrative, suitable for use in university ABM courses
- Mesa 4.0 migration guide contribution: document the 6 failure categories as a formal migration guide section in Mesa's official docs
- GeoMesa examples: fix and update all geographic examples (GeoSchelling, GeoWolfSheep) which have additional Mesa-Geo dependency complexity
- Second ownership tier: introduce a "co-owner" mechanism so examples can have a primary owner and a fallback, further reducing single-point-of-failure risk

Vanya Kapoor · vanya_kapoor1010@gmail.com · github.com/Vanya-kapoor · NSUT Delhi, India